

Case Study: Frontier Lab Intelligence

The logo for BIT Capital, featuring the text "BIT/capital." in white on a black rectangular background.

AI ENGINEER (M/F/D) · BIT CAPITAL · TAKE-HOME CASE STUDY

/ Logistics

Please check the email for the submission deadline.

After submission, there's an on-site case discussion with the AI team. We'll go deep on your design choices, trade-offs, and what you'd build next, so come ready to discuss and extend your work.

You have a €100 budget for APIs and other services, and we're interested in how you spend it. Keep your receipts and send them to us – we'll reimburse up to €100.

/ The context

You're joining the AI Engineering team at BIT Capital, a tech-focused investment fund. We invest in technology companies, including those building frontier AI, and our edge depends on staying at the frontier of what the labs are actually doing, often before it shows up in a product or a stock price.

The people who build frontier AI (researchers and engineers at OpenAI, Anthropic, Google DeepMind, Meta, xAI, Mistral, DeepSeek, Qwen, and the stealth startups they keep leaving to found) broadcast a significant amount of signal in public: papers, technical blogs, GitHub, conference talks, and especially X. So do the labs themselves, through official research blogs, model and system cards, release notes, and their org accounts. Buried in that firehose are genuinely valuable, early signals: a capability breakthrough months before it's productized, a key researcher quietly moving to a competitor, a shift in a lab's strategic direction, an architectural innovation to improve training or inference efficiencies with ramifications throughout the semiconductor value chain, etc.

We want you to build the system that tracks this.

Before you write any code, research BIT Capital using whatever is public: our website, fund documents, press, LinkedIn. Understand what we invest in and who would actually use this. Part of the task is figuring out why this intelligence matters to a fund like us, and who the system users are.

/ Your task

Build a working system that:

1. Tracks a register of who and what to follow: the frontier labs themselves (via their official channels) and the key individuals inside them.
2. Ingests and extracts intelligence from what they publish: official lab blogs and accounts, plus individuals' papers, posts, repos, talks etc.
3. Scores and ranks the contributors and the insights: this is the data-science core, and it should be justified and reproducible.
4. Delivers actionable intelligence as reports and alerts to two internal audiences (see below).

Use any AI coding tools you want. We expect you to, and we'll ask how you worked.

/ What success means

We are not looking for a vibe-coded demo that ticks the boxes. We are looking for a system that actually delivers interesting, relevant intelligence with good filtering, one that shows taste and gets the details right. The single most important question we'll ask when we open your submission is: **did this surface something we'd genuinely want to know, and did it keep the noise out?** Build toward that.

/ The two audiences

The same raw intelligence serves two internal consumers who care about overlapping but different things. Build one shared core (ingestion → register → extraction → scoring) and two tailored last-mile outputs:

- **Investment team (PMs / analysts):** "what does this mean for our positions?" e.g. a star researcher leaving a lab to found a startup; a capability jump that threatens a holding's moat or validates a thesis; shifts in the competitive map between the public names exposed to AI, implications for the semiconductor and energy supply chains. Framing: implications, tickers, theses. (Note: most labs are private, so part of the judgment is connecting lab developments to where they actually land for a public-equity investor.)
- **AI team (the people you'd be joining):** "what should we adopt or investigate?" e.g. a new agentic-orchestration or evaluation technique to bring into our own platform; a cheaper/better open model; a paper that changes how we'd build a pipeline. Framing: technical, build-relevant.

We want to see that you understand the difference and tailor the output, not that you build two separate systems.

/ The system, part by part

1 Frame the problem (who & what to track)

Decide which labs and which individuals matter, and why. How do you define "key"? Founders and research leads are obvious; the real signal is often one layer down, and a lot of it comes straight from the labs' own channels. How do you keep the register current as people move and new names emerge?

2 Ingestion pipeline

Pull from heterogeneous sources on a schedule. Official lab channels (blogs, org X accounts, etc.), papers (e.g. arXiv), GitHub, and more. The sources are messy and some are gated or expensive. We care about how you handle source selection, rate limits, deduplication, and freshness, plus how you reason about which sources are worth the effort. Decide and justify your coverage: a thoughtfully-scoped set of sources done well beats a broad set done badly.

3 The register of labs and individuals

This is the foundation: everything downstream is only as good as who and what you choose to track. Track two kinds of entity:

- The labs themselves, as first-class entities, via their official channels (research blog, org X account, publications/research index, GitHub org, model and system cards, newsroom). A lot of the highest-value signal is published by the lab, not a named person, so tracking individuals alone will miss releases, system cards, and official announcements entirely.
- The key individuals inside them: the obvious founders and research leads, but also the layer below them where much of the real signal sits.

Get the right labs and people, keep the list current as people move and new names and orgs emerge, and do the entity resolution to know that an X handle, an arXiv author name, and a GitHub account are the same person (and that a personal post and an official lab post are different kinds of source). Part automated discovery, part judgment.

4 Extraction & intelligence

For each tracked entity, lab or individual, extract the key contributions and developments over a recent window (you choose it, we'd suggest ~3 months; recency and trend matter). Turn raw content into structured, attributed insights (attributed to the person or the lab), with the LLM doing the heavy lifting. Every insight must be traceable to its primary source: cite cleanly.

5 Scoring & rating: the data-science part

Score and rate both contributors and insights. This is where we look hardest at your data-science instincts. A score is only as good as its justification: what are the inputs, why those weights, how do you know it's measuring what you claim? Validate it, against ground truth, human judgment, or a defensible proxy. An arbitrary weighted sum dressed up as a "score" is a red flag; a simple model you can defend and have tested is exactly what we want.

6 Signal vs. noise: the most important part

Most of the firehose is noise. The system must surface the genuinely important, novel, and actionable, and suppress everything else. This is where taste shows, and it's weighted accordingly. Show us your filtering logic and your judgment.

7 Actionable intelligence: reports + alerts

Deliver the goods to both audiences:

- **Reports:** a periodic digest (you choose the cadence) readable in the app and exportable as PDF.
- **Alerts:** when something material appears, push it (e.g. Slack/email) to the right audience.

Both must cite sources and be tailored per persona (see above). "Actionable" means a reader knows what it means and what to do, not just a summary of what happened.

8 Web interface

A small surface where a user can browse the register, see the scored insights and why they were flagged, read past reports, and configure what's tracked. Keep it light and quick. Don't spend too much time polishing the UI.

/ How to prioritize

The case study is deliberately ambitious. We've set a high ceiling on purpose, because we want to see how far you can take it. Aim to complete it all – but know that if you don't, that's not automatically a failure. In that case, we prefer to see a few sections done in depth, rather than all sections done superficially in an attempt to "tick all boxes". To help you prioritize, we are providing the table below. It's a rough ranking of what's important for us (in descending order). We suggest that you start at the top and work your way down.

AREA	WEIGHT
Registry of labs and individuals: the right entities (including the layer below the obvious names), comprehensive, entity-resolved, and kept current. The foundation everything builds on.	20%
Signal-vs-noise filtering & quality of the intelligence	20%
Scoring / rating of contributors & insights: data-science rigor + validation	20%
Actionable intelligence delivered: reports + alerts, two personas, cited	15%
Ingestion pipeline: scheduled, multi-source, robust	10%
Extraction into structured, attributed, cited insights	10%
Web interface	5%

/ Deliverables: what we'll be looking at

- **Source code in a Git repo:** clear, modular, good primitives (clean functions for ingestion, extraction, scoring, storage, orchestration, delivery). README + a demo we can run locally (or a live deployment).
- **The database:** schema and real data.
- **Architecture write-up:** the stack and why, including model selection per task (which model for which job, and why) and fallback strategies.
- **Prompts** with your design rationale.
- **Evaluation:** how you measured extraction quality and controlled hallucination, and how you validated the scoring. Show your ground-truth approach.
- **Tokenomics:** approximate token usage and \$ cost per workflow, and how cost shaped your model choices (cost-quality trade-offs).
- **Final report:** what works, what you'd do next, what you learned, and the 3–5 most interesting real insights your system actually surfaced (this is your proof that it works).
- **Optional:** a short Loom / video demo.